

File-driven scrollback for the Ink UI

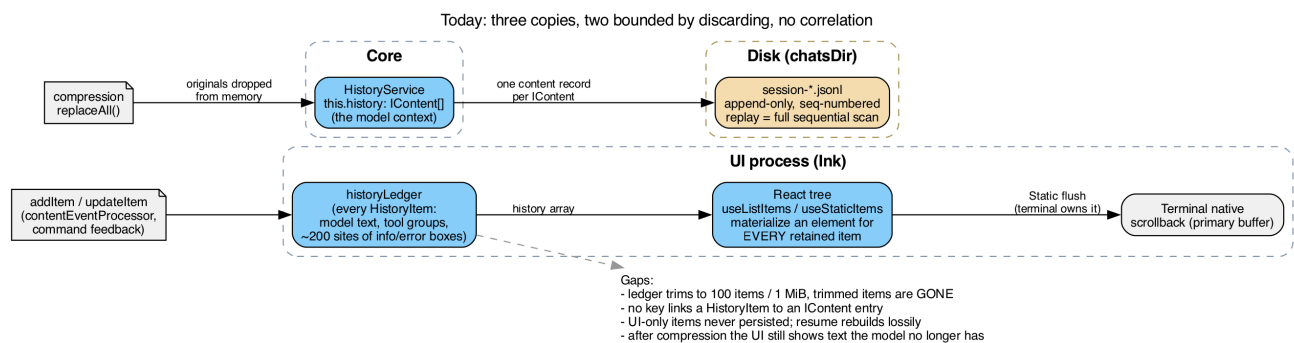
Design outline for issue #854 · [vybestack/llxpri-code](#) · branch `issue854` · milestone 0.12.0 · labels: Ink UI, Context Management

Summary. Scrollback today is bounded by discarding: the UI ledger trims to `ui.historyMaxItems/`
`ui.historyMaxBytes` (effective defaults 100 items / 1 MiB) and everything trimmed is gone, while the model keeps a different copy in `HistoryService` and compression silently drops the originals from memory. This design makes scrollback a **windowed view over an append-only on-disk journal**: memory holds the visible viewport, a margin, and the live turn; everything else, including everything compressed out of context, lives on disk and pages in when the user scrolls back, and drops out again when they scroll forward. Items are marked "in context" vs "purged", and each compression boundary gets an expandable summary row. The model-side context stays in memory until phase 5, which then deletes that copy: `HistoryService` becomes a facade over the same journal — per model call, rows stream from the file through a transformer with nothing held in memory (section 2b).

One journal per session, one writer per journal, three kinds of reader. The agent loop's content commits write the session journal; the UI scrollbar (via JournalCursor), the model context (HistoryService facade), and resume replay read it. Subagents use the same disk-based path: each run gets its own journal file and the same facade over it. No runtime holds an in-memory copy of the context.

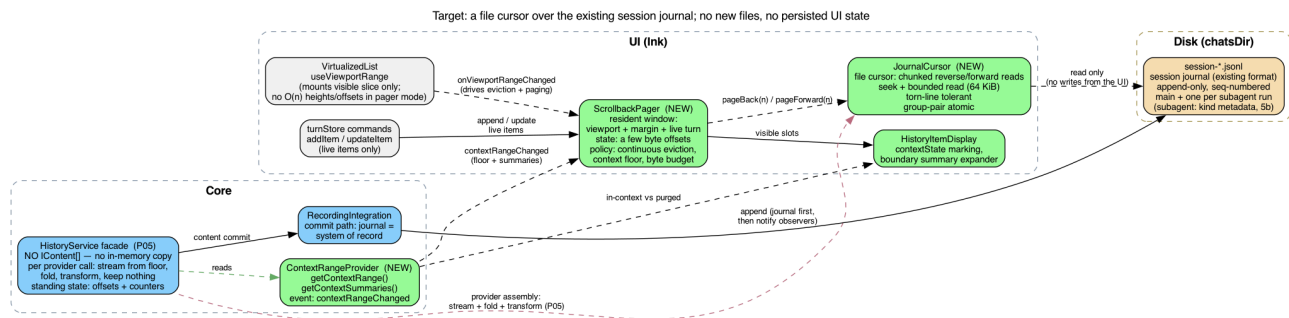
1. What exists today (verified on main)

Three copies of the conversation exist. The UI ledger (`historyLedger.ts`) holds every `HistoryItem`, including ~200 call sites of UI-only info/error boxes, trimmed to 100 items / 1 MiB (then discarded). `HistoryService` holds `IContent[]`, what the model actually sees; compression `replaceAll()` drops the originals from memory. The append-only session journal (`session-*.jsonl`) records content additions and compression/rewind events, but has no offsets and is only replayed sequentially. There is no correlation key between UI items and context entries, no query for "what is in context", UI-only items are persisted nowhere, and resume rebuilds the transcript lossily. OpenTUI is gone from main, so the Ink UI is the only target.



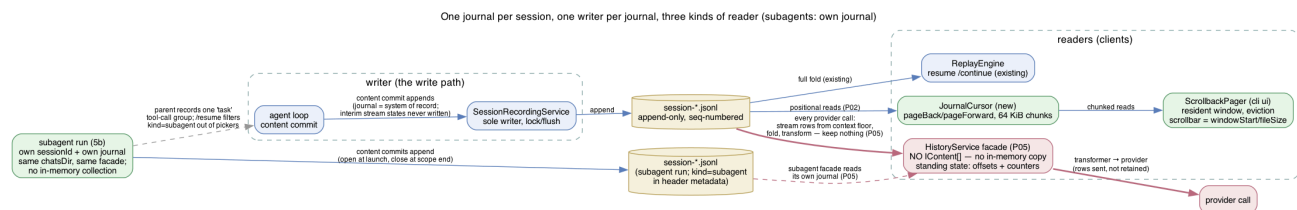
2. Target architecture

Per Andrew's three rulings (2026-09-18): no duplicate file, no persisted UI state of any kind, and no giant in-memory map — "the point is to scroll up and down the file, not have shit in memory. goal is LOW memory usage and high eviction without functionality loss." The ledger becomes a **ScrollbackPager** that scrolls `session-*.jsonl` directly through a **JournalCursor**. A **ContextRangeProvider** added to **HistoryService** (`getContextWindow()`, `getContextSummaries()`, `event contextRangeChanged`) supplies the context floor. Every live UI item is stamped with the `chronology.seq` of the context entry it derives from, so "is the model still seeing this?" is one integer comparison at render time.



Journal clients and package responsibilities

The map below is the whole architecture in one picture: the main agent loop is the **writer** — a content commit appends to `session-*.jsonl`; the journal is the system of record, not a mirror of memory. Three **readers**: the UI scrollback (**JournalCursor**, phase 2), the model context (the **HistoryService facade**, phase 5), and the existing **ReplayEngine** at resume. Each subagent run writes **its own journal** in the same `chatsDir` and gets the same facade over it.



Component responsibilities

- **Session journal (existing, unchanged):** the only durable artifact. The existing recording path is the only writer; format byte-for-byte identical; discovery and janitor globs untouched.
- **JournalCursor (new, core recording):** chunked reverse/forward reads over the journal. `pageBack(n)` seeks to `max(0, windowStart - CHUNK)`, reads through `windowStart`, drops the torn head partial line, parses records. The journal is append-only and seq-numbered, so reverse byte order IS reverse chronology. Default `CHUNK` 64 KiB. Tool groups span exactly 2 adjacent records; a chunk boundary splitting a pair triggers one extra bounded read (group atomicity). Crash-torn trailing lines are ignored on parse, matching `/continue`. Nothing is ever written, built, or repaired.

- **ScrollbackPager (new, cli ui)**: resident window + a handful of byte offsets. Owns the eviction and paging policy (sections 3-4). Memory is constant in session length.
- **ContextRangeProvider (new, core)**: exposes the in-context first/last seq, the summaries with their replaced spans, and emits `contextRangeChanged` after compression, rewind/clear, and first add. Under the P05 facade the same queries resolve from journal state, so the UI contract does not change when the backing store flips.
- **Correlation stamping**: `HistoryItem.chronologySeq?` and `seqSpan?`, stamped at creation in `contentEventProcessor`, user echo, tool-group assembly, and the resume fallback converter. In-memory ids only, per ruling 2. Verified during design: a standard tool group spans exactly 2 adjacent `IContent` entries with interleaving structurally excluded by the serialized loop; replay reconstruction by seq order + callId join is already deterministic.

2b. Streaming, subagents, the facade, and the scroll buffer (answered from the code)

Realtime streaming is untouched by construction

Stream deltas flow through the existing hot path: `useAgentStream` accumulates partial items and calls `setPendingHistoryItems` per delta; the layout renders the pending array below the pager's committed rows. No disk read, no disk write, no cursor involvement. The journal sees only commits: when a turn's content finalizes, the committed entries append to the journal and the rows cross from "pending" to "resident tail" in one step. Streaming cost stays $O(\text{current turn})$ regardless of scroll position, and a subagent's streamed `<subagent>` text is just the live `task` tool call's output on the same path.

Subagents: same disk-based path, own journal files (ruling 2026-09-18)

Verified today: every agent runtime builds its own in-memory `HistoryService` (`createAgentRuntimeContext`), and only the interactive CLI session creates a `RecordingIntegration` (`cliSessionBootstrap`, `performResume`) — that per-runtime in-memory collection is exactly what this design removes. Per the ruling, subagents use the same disk-based path: each run gets its own `sessionId` and its own `session-*.jsonl` in the same `chatsDir`, opened at launch and closed at scope end. The header's existing `sessionMetadata` record carries `kind: "subagent"` plus `parentSessionId`, so `/resume` and pickers filter them out of human-facing lists while discovery globs and the janitor stay untouched (they already match `session-*.jsonl` uniformly). The subagent's `HistoryService` is the same facade over its own file. The parent journal still records exactly one `task` tool-call group per run.

HistoryService becomes a facade: no in-memory copy, period

Andrew's acceptance criterion: "there must be no in-memory copy of the context in the `HistoryService`; it will instead be a facade over the file. when it's time to send to the model the rows from the file are sent through a transformer to the model with nothing being held in memory." What dies: the `protected history: IContent[]` array in `HistoryServiceCore` and compression's array swap — there is no array to swap. What replaces it: content commits append to the journal first (the write path; observers are notified after), and every provider call is a streaming pass — open the cursor at the context floor (the byte offset of the last compression boundary, or 0), read records forward, fold (append; at a `compressed record` drop the replaced span and continue after the summary; honor rewind truncation), transform survivors to provider messages, send. When the call returns, the streamed rows are garbage; nothing is cached across calls. The only standing state is $O(1)$ and contains no content: context-floor byte offset, tail offset, last seq, a token-estimate counter, and pointers (`seq + offsets`) to compression summaries whose text is re-read on demand. Speed: each model call re-reads the in-context region — local sequential reads of an append-only file the kernel page cache already holds (the writer just wrote it); microseconds to low milliseconds, and the page cache is kernel memory, evictable under pressure, not a copy we own. Acceptance tests: (1) no collection proportional to context length anywhere in `HistoryService` (structural); (2) retained heap after GC does not grow with N turns (property); (3) streaming assembly matches the full event fold byte-for-byte, including compression and rewind (property); (4) subagent facades pass the same criteria; (5) `/continue` resumes with no full-materialization step anywhere.

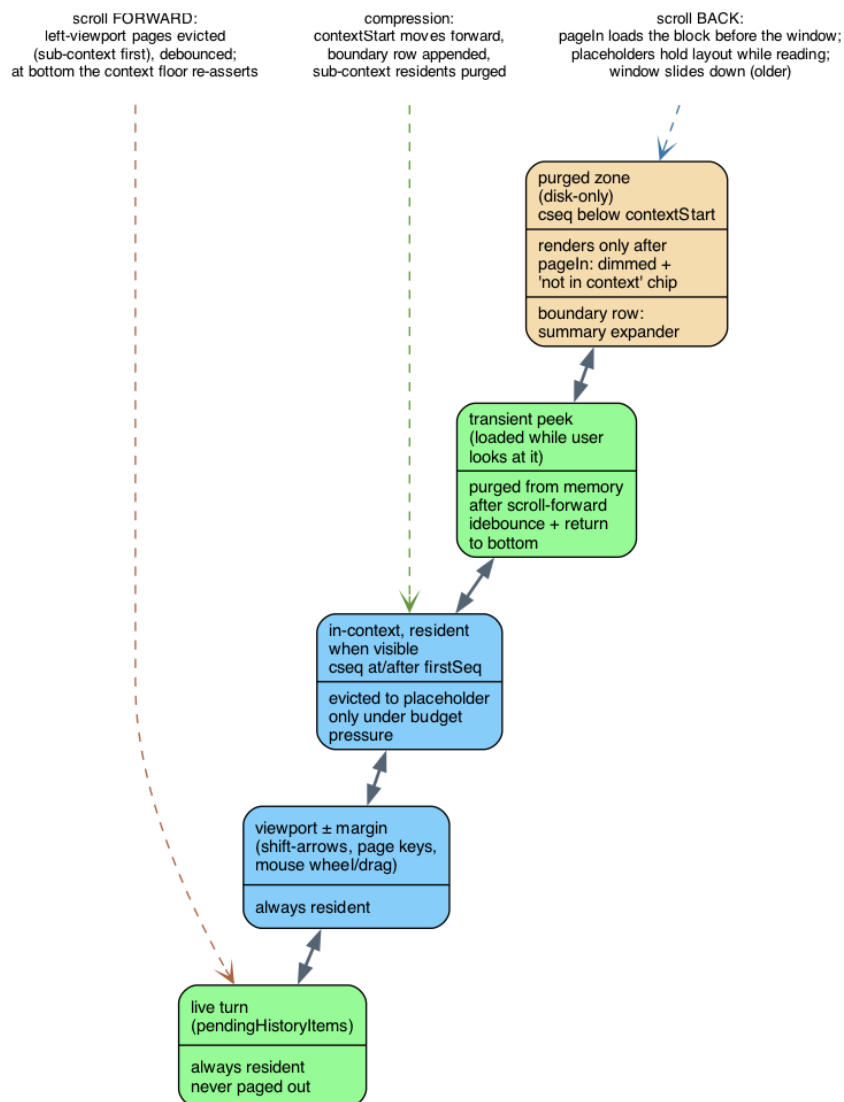
What React holds in the scroll buffer

"Is this a giant ass tree that never leaves memory?" No — and the current code already proves it. The committed transcript in alternate-buffer mode renders through `ScrollableList` → `VirtualizedList`, whose `useViewportRange` computes a `startIndex..endIndex` slice; `useRenderedItems` creates elements for that slice only, between two spacer boxes. Rows outside the viewport unmount: their elements, fibers, and Ink/Yoga layout nodes become garbage. React keeps no element archive; a re-render recreates $O(\text{viewport})$ elements, not $O(\text{session})$. The real $O(n)$ offenders today are (1) the `data` array — currently the ledger capped at 100 items / 1 MiB, which the pager replaces with the resident window's slots (`viewport + margin + pending`); and (2) `VirtualizedList`'s `heights/offsets` number arrays, which accumulate one entry per ever-mounted item — pager mode does not reuse them: the scrollbar is byte-proportional (`windowStart / fileSize`) and heights are measured for in-viewport slots only, dropped on evict. Primary-buffer mode prints committed rows once through `Ink Static`; the terminal owns those bytes afterward. Net transcript working set: `viewport + margin + current turn + a few byte offsets + one 64 KiB read chunk`. Nothing scales with session length.

3. The sliding window

Residency is one contiguous range over the timeline. Always resident: the live turn (pending items). The viewport keeps a margin (proposal: two viewport heights) resident around it. At bottom (steady state), anything with `cseq` below the context start is evicted as soon as it is not visible: the issue's "trim anything not in HistoryService" rule. Scrolling back beyond the window pages the previous block in from disk (placeholders hold layout, since the virtualized list already paints from estimated heights). Scrolling forward evicts the pages that left the viewport, sub-context items first, debounced ~1-2s after scrolling stops. A hard byte/item budget (proposal: retarget `ui.historyMaxBytes/ui.historyMaxItems` from "display trim" to "resident window") evicts from the far end regardless.

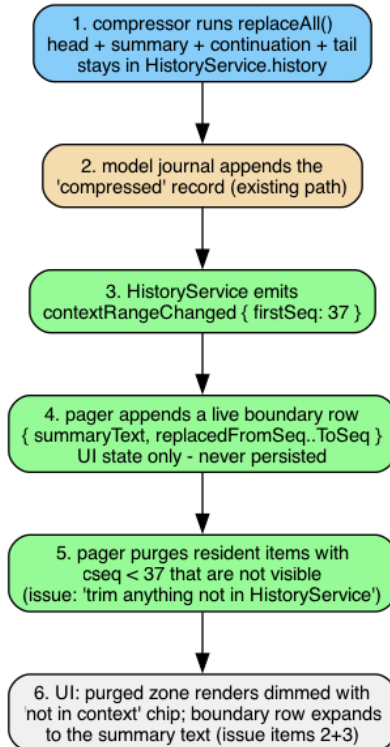
Sliding window over `uiSeq` (one contiguous resident range; everything else is a placeholder)



4. Key flows

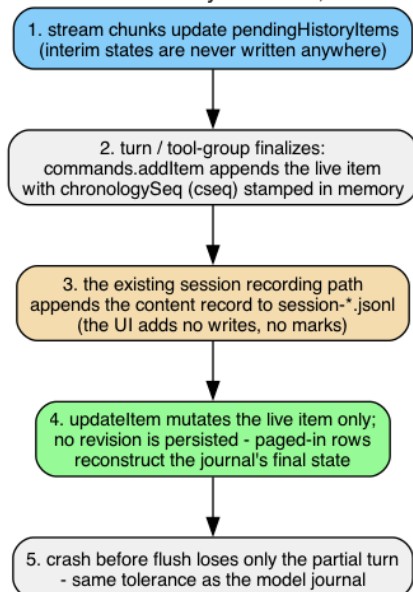
Compression (the money flow)

Compression: boundary moves, residents purge, summary becomes expandable



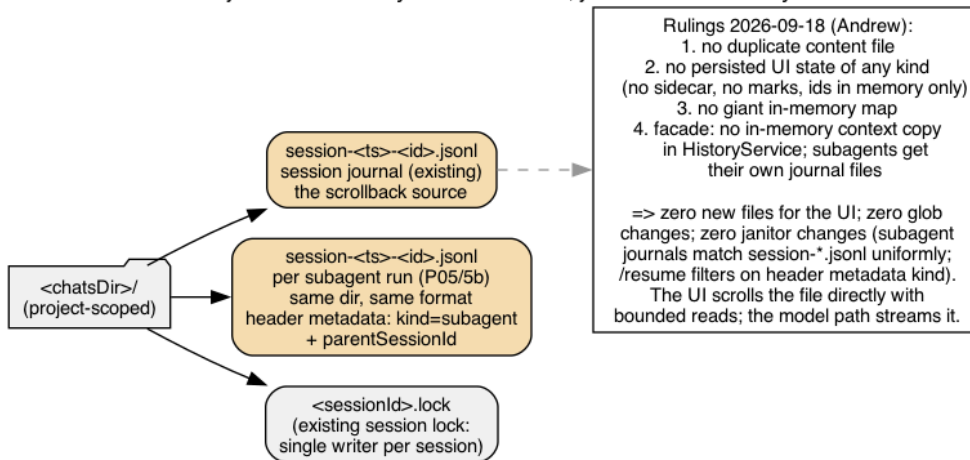
Turn append

Turn append: streamed text stays resident; the UI writes nothing



5. Files, naming, janitor

Files: the session journal is the only durable artifact; janitor and discovery are untouched



Zero new files. `session-*.jsonl` is the only durable artifact; the scrollbar is a read-only view over it. Because nothing is added to `<chatsDir>/`, every discovery/janitor/cleanup glob (matches exactly `startsWith('session-') && endsWith('.jsonl')`) and the janitor's archive grouping work unchanged. Single writer per session is already enforced by the existing session lock; the UI never writes.

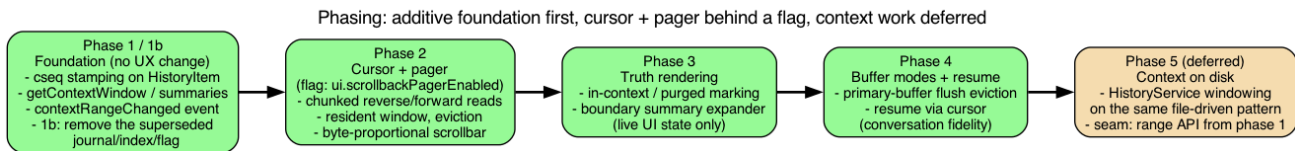
6. Rendering and buffer modes

- `HistoryItemDisplay` takes a derived `contextState`: **in-context** (unchanged, optional left rail), **purged** (dimmed + "not in context" chip), **n/a** for UI-only items (no marking).
- Compression boundary row: collapsed shows "compressed N messages"; expands to the summary text from the journal/context summaries (issue items 2 and 3).
- **Alternate buffer** (shift-arrows, page keys, mouse wheel/drag): full pager behavior; this is where the feature lives.
- **Primary buffer** (no in-app scroll; Ink Static flushes rows once and the terminal owns them): eviction-on-flush. Once Static prints an item it is dropped from the ledger; remounts after a clear reprint only resident items plus a one-line notice. Paging does not apply; the terminal's own buffer is the scrollbar there.
- **Resume**: open a file cursor on `session-*.jsonl`; resident = context-range items + last viewport page, older rows chunk-read backward on demand. Fidelity is conversation-exact — what / `continue` reconstructs, with seqs stamped. UI-only rows (banners, toasts, panels) do not exist after restart: accepted property of the no-UI-state ruling, same as today.

7. Memory outcome

Before: ledger capped at 1 MiB by discarding, a React element materialized for every retained item on every history change, and — the big one — the `HistoryService IContent[]`: a full uncompressed copy of the context held for the whole process lifetime, once per runtime (main session plus every subagent). After: resident items are viewport + margin + live turn, one 64 KiB chunk buffer during a page read, and a few byte offsets of cursor state; after P05 the facade's standing state is the same shape — $O(1)$ offsets and counters, zero context rows. **Nothing scales with session length**, in the UI or the model path. Elements exist only for resident slots, and scrollbar and context are unbounded on disk instead of held or discarded.

8. Phasing



- **Phase 1 - foundation, no UX change:** cseq stamping; range API and event in core. **Phase 1b rework** (per the rulings): remove the superseded journal/index writer, its wiring, and `ui.scrollbackJournalEnabled`.
- **Phase 2 - cursor + pager** behind `ui.scrollbackPagerEnabled`: chunked reverse/forward reads, residency, viewport feedback, budget + context-floor eviction, byte-proportional scrollbar, scroll-back paging.
- **Phase 3 - truth rendering:** in-context/purged marking, boundary expander (live UI state only).
- **Phase 4 - buffer modes + resume:** primary-buffer flush eviction; cursor-based resume (conversation fidelity).
- **Phase 5 - facade flip** (sections 2b/5b/5c): delete the in-memory `IContent[]` from HistoryService; provider calls stream from the journal (cursor at the context floor → event fold → transformer → send, nothing retained); subagent runs get their own journals with `kind: "subagent"` metadata and the same facade; `/resume` filters on that metadata. Acceptance criteria in section 2b. The model and the UI are two clients of one disk-backed truth; the range API and JournalCursor are the seam, no UI changes required.

9. Open questions — status after review

Resolved 2026-09-18 with Andrew on PR #3727:

1. Static-remount after flush eviction reprints the **resident set only**; older content pages in from disk just-in-time on scroll. No larger primary-buffer tail.
2. Primary-buffer (print-through) mode: the terminal emulator's own scrollbar is the look-back viewer; we evict our copy after printing and never re-render. Paging applies to alternate-screen mode only.
3. Budgets are **row-based, not item-count-based**: resident = viewport + 2 viewports of margin plus a byte floor so huge items cannot starve the window. Tunable via settings once introduced.
4. Sparse on-disk index: **deferred, then made moot** by ruling 3 — there is no index at all (no file, no in-memory map), so there is nothing to sparsify.
5. `updateItem`: on-screen items always update in place immediately; with no persisted UI state there is no revision record — off-screen items need no handling, and paged-in rows reconstruct the session journal's final state. Eviction is continuous as items leave viewport+margin.
6. Scrollbar: **byte-proportional** (`windowStart / fileSize`) per ruling 3 — height bookkeeping beyond the visible viewport is unnecessary; measured heights are used in-viewport only, never persisted.
7. `pageIn` vs `clear` markers: **hard stop** at the oldest in-context item; no "show cleared history" affordance. A full activity journal regardless of compression is a possible later feature, out of scope.

OQ8 — sidecar strategy: RESOLVED by Andrew's three rulings (2026-09-18). (1) The duplicate file is specifically rejected: no `sb-` copy of conversation content and no persistent index file. (2) NO UI state is persisted at all — nothing about what is being displayed, has been displayed, or was evicted; ids and pointers in memory are fine, marks in files are not; there is no sidecar of any kind, and UI-only rows are live-only. (3) No giant map in memory either — the point is to scroll up and down the file; LOW memory usage and high eviction without functionality loss. The pager is a file cursor with chunked reads; memory is constant in session length.

OQ9 - review gaps raised by Andrew 2026-09-18: RESOLVED, then partially superseded by OQ10. (1) Subagents and the writer are now in the diagrams and the map (agent loop writes; subagents per 2b/OQ10). (2) Realtime streaming is preserved by construction: the pending region is live memory on the existing hot path and the journal sees only commits (section 2b). (3) Context is direct-from-journal to the model — the projection-with-in-process-feed idea originally proposed here was **rejected** by OQ10; the facade holds no copy at all.

OQ10 - facade ruling, RESOLVED by Andrew 2026-09-18: (1) *"there must be no in-memory copy of the context in the HistoryService; it will instead be a facade over the file. when it's time to send to the model the rows from the file are sent through a transformer to the model with nothing being held*

in memory." Acceptance criteria in section 2b. (2) *"subagents need to use the same disk-based path... this means subagents will need their own files (and that's fine, they should)"* — own journal per run, same facade. (3) Scroll buffer: *"is this a giant ass tree that never leaves memory?"* Answered in 2b from the code — React mounts only the visible window; unmounted rows (elements, fibers, Yoga nodes) are collectible; the $O(n)$ offenders (ledger data array, heights/offsets arrays) are replaced by the resident window and a byte-proportional scrollbar.

Resolved during design: tool-group `seqSpan` well-definedness was verified against the code (a standard group = 2 adjacent entries, no interleaving, write-once commit, deterministic replay by seq order + callId join). Section 2 carries the facts.

10. Source

Full design text: `project-plans/issue854/issue-854-design.md` (branch `issue854`). Diagrams: graphviz sources under `tmp/verify854/diagrams/` (regenerable).